Checkpoint NLP 3 · Clasificación de noticias con DistilBERT + LoRA

Data Science III: NLP & Deep Learning · Módulo 4 · Pre-entrega 4

Elizabeth Luque · Coderhouse · noviembre 2026 · repositorio:
github.com/scarletluque0805/ds3-transformer-lora

Resumen en una mirada

	A · TF-IDF + Naive Bayes (Módulo 3)	B · DistilBERT + LoRA (Módulo 4)
F1-macro en test (2.000 noticias)	0,8920	**0,9025** (+1,04 pts)
Parámetros que entreno	103.120	741.124 (1,11% de 66.956.548)
Tiempo de entrenamiento	0,44 s en CPU	≈ 1 min en GPU Tesla T4 (58,8 a 63,2 s en 3 ejecuciones)
¿La mejora es significativa?	—	No al 5%: McNemar p = 0,115

1. Resumen de arquitectura

En esta pre-entrega quise ver si un Transformer preentrenado, ajustado con **LoRA**, le gana a mi baseline clásico del Módulo 3 en la misma tarea: clasificar noticias de **AG News** en 4 categorías (Business, Sci_Tech, Sports, World). Para que la comparación fuera justa usé **exactamente el mismo test**: las 2.000 noticias de *ag_news_test.csv*, en el mismo orden, evaluadas una sola vez al final. Del train original separé un 10% estratificado como validación (7.200 / 800, *seed* 42) para elegir la mejor época sin mirar el test.

1.1 Pipeline de punta a punta

Etapa	Qué hago	Dónde está
Texto	Limpieza estructural del Módulo 2 (entidades HTML, URLs, firmas de agencia, datelines). **No lematizo ni saco stop-words**: el Transformer necesita el texto natural y su contexto.	src/limpieza.py
Tokenización	WordPiece de distilbert-base-uncased, *max_length* 72, padding dinámico	src/entrenamiento.py
Encoder	DistilBERT de 6 capas, **congelado**. Solo agrego adaptadores LoRA en Query y Value de la atención	src/modelos.py
Cabezal	pre_classifier + classifier (768 → 4), nuevos y entrenables	src/modelos.py
Entrenamiento	Trainer de Hugging Face, 4 épocas, me quedo con la mejor época según F1-macro de validación	notebook Parte 2
Evaluación	Test una sola vez, benchmark contra el Módulo 3, McNemar y matrices de confusión	notebook Parte 3

1.2 Por qué elegí distilbert-base-uncased

Comparé cuatro candidatos del Hub de Hugging Face. Calculé los parámetros desde la configuración de cada modelo, con la misma LoRA (r = 8 sobre Query y Value):

Modelo	Idioma	Capas	Parámetros	Entrenables con LoRA
distilbert-base-uncased (elegido)	Inglés	6	66,96 M	741.124 (1,11%)
bert-base-uncased	Inglés	12	109,49 M	297.988 (0,27%)
roberta-base	Inglés	12	124,65 M	888.580 (0,71%)
distilbert-base-multilingual-cased	104 idiomas	6	135,33 M	741.124 (0,55%)

Me decidí por tres razones. **Idioma:** el corpus está en inglés, y el multilingüe gasta el 68% de sus parámetros en embeddings de 104 idiomas que no necesito. **Presupuesto:** con 6 capas tiene la mitad de cómputo que BERT o RoBERTa, conserva alrededor del 97% de su desempeño y entra cómodo en la T4 gratuita de Colab. **Tarea:** son 4 temas con 7.200 ejemplos, una tarea de comprensión que no requiere un modelo grande. A

roberta-base lo dejé como techo de referencia y lo entrené igual para medirlo (sección 3.3).

1.3 Convenciones de tokenización, padding y truncado

Las dejo explícitas porque en la devolución de la Pre-entrega 2 me pidieron precisarlas:

Aspecto	Convención
Tokens especiales	Cada noticia queda `[CLS] … [SEP]`. El largo que medí para el p95 **ya incluye esos 2 tokens**, así que 72 es el largo total que recibe el modelo.
Cómo medí el largo	`len(input_ids)` de cada noticia del train sin truncar: es un valor por documento, no un conteo de ocurrencias. Mediana 48, p95 70 → redondeé a 72 (múltiplo de 8). Así el **95,7%** de las noticias entra completo.
Truncado	`truncation=True, max_length=72`. Se recorta el final del cuerpo; el titular, que es lo más informativo, queda siempre.
Padding	Dinámico con `DataCollatorWithPadding`: relleno con `[PAD]` (id 0) solo hasta la noticia más larga de cada lote, y la *attention mask* pone 0 en esas posiciones.

2. Configuración PEFT

La idea de LoRA es no tocar los pesos originales W y aprender solo una corrección de bajo rango, $\Delta W = B \cdot A$, escalada por α/r. Con r = 8, cada matriz de atención de 768×768 (589.824 pesos) se ajusta con apenas 2 × 768 × 8 = 12.288 parámetros. Uso `peft` con `task_type=SEQ_CLS`:

Hiperparámetro	Valor	Por qué lo elegí
r (rango)	`8`	La tarea es simple y está cerca del preentrenamiento, así que alcanza con un rango bajo. Lo validé con una ablación 4 / 8 / 16.
lora_alpha	`16`	Escala α/r = 2, la convención estándar.
target_modules	`q_lin, v_lin`	Query y Value en las 6 capas: según el paper de LoRA, es la mejor relación entre desempeño y parámetros.
lora_dropout	`0,1`	Regulariza los adaptadores.
modules_to_save	`pre_classifier, classifier`	El cabezal es nuevo (4 clases), así que tiene que entrenarse completo y guardarse con el adaptador.

2.1 Parámetros entrenables vs. totales

Bloque	Parámetros	% del modelo
Modelo original (DistilBERT + cabezal)	66.956.548	100%
Adaptadores LoRA (A y B en Q y V, 6 capas)	147.456	0,22%
Cabezal de clasificación (pre_classifier + classifier)	593.668	0,89%
Total entrenable	**741.124**	**1,11%** (requisito: < 3%)
Congelado	66.215.424	98,89%

Verifiqué que el Transformer base queda congelado y que las matrices B arrancan en cero, así que al inicio ΔW = 0 y el modelo parte exactamente del preentrenado. El Trainer informa 67.697.672 parámetros en total porque `modules_to_save` guarda una copia entrenable del cabezal junto a la original: 66.956.548 + 741.124. El adaptador final pesa **3,0 MB**, contra 268 MB del modelo base.

2.2 Argumentos de entrenamiento

Argumento	Valor	Argumento	Valor
`learning_rate`	3e-4	`per_device_train_batch_size`	32
`num_train_epochs`	4	`weight_decay`	0,01
`warmup_ratio`	0,06	`fp16`	True (GPU)
`eval / save_strategy`	epoch	`load_best_model_at_end`	True
`metric_for_best_model`	f1_macro	`seed`	42

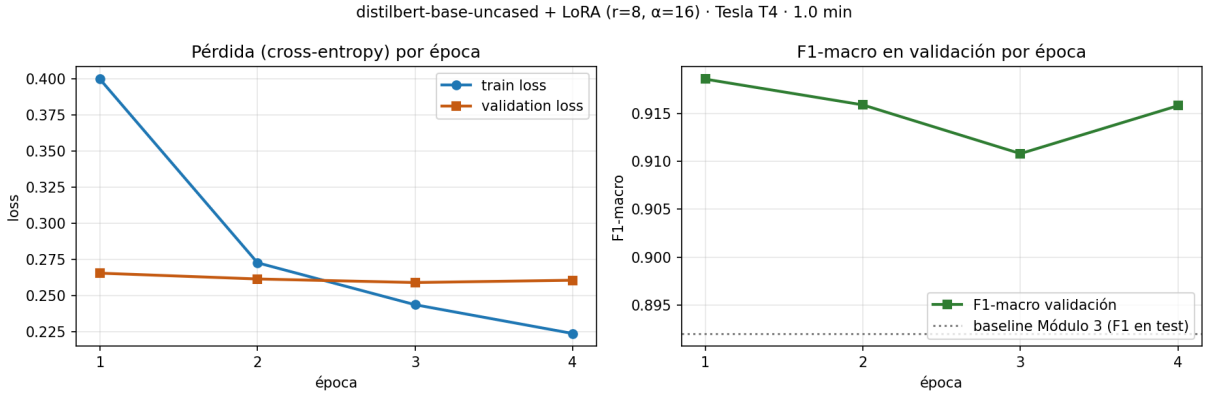
El learning rate es 10 veces más alto que el típico de un fine-tuning completo (2e-5 a 3e-5). Con el modelo base congelado no hay riesgo de *catastrophic forgetting*, y los adaptadores, que arrancan en cero, necesitan

pasos más grandes. Lo comprobé en la ablación: con 3e-5 el modelo aprende bastante menos.

3. Resultados y comparativa

3.1 Entrenamiento: pérdida de train vs. validación

Época	train loss	val loss	val F1-macro	val accuracy
1	0,4000	0,2656	0,9186	0,9187
2	0,2728	0,2616	0,9159	0,9163
3	0,2438	0,2591	0,9108	0,9113
4	0,2238	0,2607	0,9159	0,9163



Corrida principal: distilbert-base-uncased + LoRA (r = 8, α = 16) en Tesla T4.

La pérdida de train baja de forma sostenida (0,400 → 0,224), mientras que la de validación queda prácticamente plana en torno a 0,26. En la época 1 el train loss supera al de validación porque es un promedio de toda la época, incluidos los primeros pasos en los que el modelo todavía no sabía nada, mientras que la validación se mide al final. Desde la época 2 se abre una brecha chica (sobreajuste leve, sin subas mayores a 0,01 en validación). El mejor F1-macro de validación lo da la **época 1 (0,9186)**. Por eso usé `load_best_model_at_end`: el adaptador guardado es el de esa época y no el de la última.

3.2 Tiempos de ejecución

Corrí el notebook de la Parte 2 **tres veces** en Google Colab, con GPU **Tesla T4** (15,6 GB) y torch 2.14.1, transformers 5.22.1, peft 0.21.2 y datasets 4.8.5. Estos son los minutos de `trainer.train()` de cada corrida (4 épocas cada una):

Corrida	1ª ejecución	2ª ejecución	3ª ejecución	VRAM pico
principal	1,03 min	0,98 min	1,05 min	0,76 GB
lr_3e-5	0,86 min	0,84 min	0,97 min	0,76 GB
r4	0,77 min	0,77 min	0,85 min	0,75 GB
r16	0,77 min	0,78 min	0,78 min	0,76 GB
roberta_r8	1,47 min	1,48 min	1,49 min	1,46 GB
Total entrenamiento	**4,90 min**	**4,85 min**	**5,14 min**	

La corrida principal tardó **61,8 s, 58,8 s y 63,2 s**: alrededor de 1 minuto, con un pico de solo 0,76 GB de VRAM. Todas las corridas juntas suman unos 5 minutos de GPU. La tercera ejecución, completa (instalación, descarga de modelos y las 5 corridas), la hice de 11:12 a 11:17. roberta-base, con el doble de capas, tardó 89 s (≈ 1,5 min) y usó el doble de memoria (1,46 GB). La evaluación de la Parte 3 también la corrí en la T4: cargar el modelo base y el adaptador tomó 21 s y **predecir las 2.000 noticias del test, 1,95 s**. La primera vez que lo intenté, Colab no me asignó GPU ("No hay ningún backend disponible"), así que lo corrí un rato después.

```
Tiempo de entrenamiento: 63.2 s (1.05 min) en Tesla T4 | VRAM pico: 0.76 GB
Parámetros: 741,124 entrenables de 66,956,548 (1.11%)

--- ESTADÍSTICAS POR ÉPOCA (train vs. validación) ---
 epoch  train_loss  val_loss  val_f1_macro  val_accuracy
   1.0      0.4000    0.2656        0.9186        0.9188
   2.0      0.2728    0.2616        0.9159        0.9162
   3.0      0.2438    0.2591        0.9108        0.9112
   4.0      0.2238    0.2607        0.9159        0.9162
```

Captura de la 3ª ejecución: tiempo, parámetros y estadísticas por época de la corrida principal.

```
{'train_runtime': '89.06', 'train_samples_per_second': '323.4', 'train_steps_per_second': '10.11', 'train_loss': '0.297', 'epoch': '4'}
{'eval_loss': '0.2211', 'eval_accuracy': '0.9263', 'eval_precision_macro': '0.926', 'eval_recall_macro': '0.9263', 'eval_f1_macro': '0.926', 'eva
roberta_r8 -> {'corrida': 'roberta_r8', 'modelo': 'roberta-base', 'r': 8, 'alpha': 16, 'lr': 0.0003, '% entrenable': 0.71, 'train_loss final': 0.
```

Captura del log de roberta-base: train_runtime de 89,06 s.

```
Dispositivo: Tesla T4 | torch 2.14.1+cu130 | transformers 5.22.1 | peft 0.21.2
```

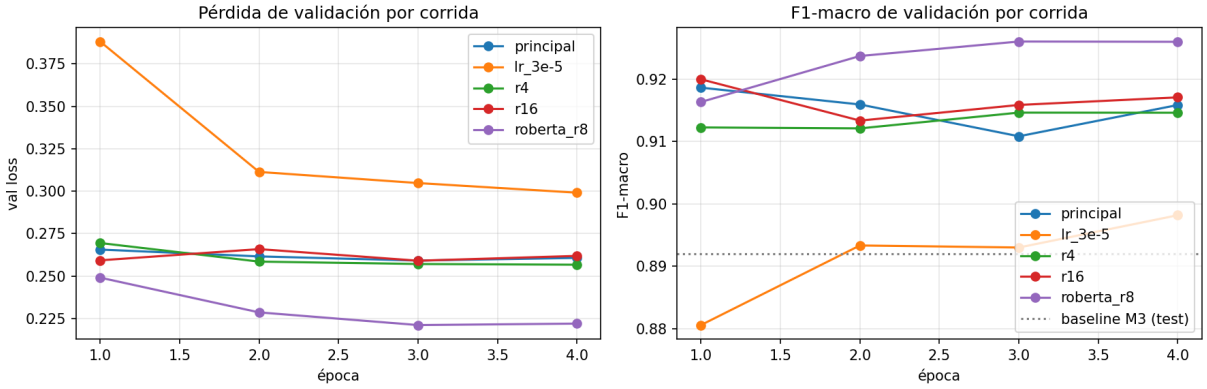
Parte 3: entorno de la evaluación.

```
model.safetensors: reconstructing file: 100%  268MB /  268MB, 25.2MB/s
model.safetensors: downloading bytes:         250MB, 22.6MB/s
Loading weights: 100%                         100/100 [00:00<00:00, 4080.82it/s]
Adaptador: ['adapter_config.json', 'README.md', 'adapter_model.safetensors'] (3.0 MB)
Parámetros cargados desde el adaptador (LoRA + cabezal): 741,124 de 66,956,548
```

Parte 3: se carga el adaptador de 3,0 MB sobre el modelo base (741.124 parámetros recuperados).

3.3 Ablación: learning rate, rango r y techo con roberta-base

Corrida	Modelo	r / α	lr	% entrenable	val loss final	val F1 (mejor)	mejor época
principal	distilbert	8 / 16	3e-4	1,11%	0,2607	0,9186	1
lr_3e-5	distilbert	8 / 16	3e-5	1,11%	0,2991	0,8982	4
r4	distilbert	4 / 8	3e-4	1,00%	0,2567	0,9146	3
r16	distilbert	16 / 32	3e-4	1,33%	0,2619	0,9200	1
roberta_r8	roberta	8 / 16	3e-4	0,71%	0,2220	0,9260	3



Pérdida y F1-macro de validación por época en cada corrida. La línea punteada es el F1 del baseline en test.

- **Learning rate:** con 3e-5, el valor de un fine-tuning completo, el modelo apenas llega a 0,898 y la pérdida sigue alta. Con LoRA hacen falta pasos más grandes.
- **Rango r:** r = 4, 8 y 16 dan 0,915, 0,919 y 0,920. Cuadruplicar el rango agrega 0,14 puntos: la corrección que necesita la tarea es de rango bajo. r = 8 queda en el punto justo.
- **Techo:** roberta-base llega a 0,926 en validación (+0,7 pts), pero tarda un 50% más y usa el doble de memoria. Lo dejé como referencia y no como modelo final, para que el benchmark sea con el modelo que justifiqué en el paso 2.

3.4 Reproducibilidad

Las tres ejecuciones dieron **métricas idénticas hasta el cuarto decimal** en las 5 corridas (por ejemplo, F1 de validación 0,9186 en la principal); solo cambian los tiempos. Fijo la semilla 42 en el split, en el Trainer y en la inicialización de LoRA. Además, en la Parte 3 cargué el adaptador guardado en un entorno nuevo y volví a

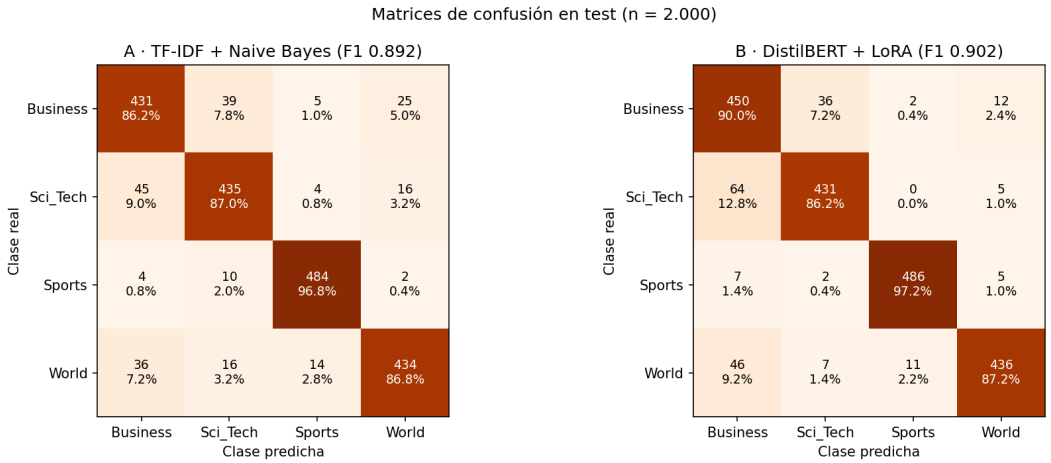
medir la validación: dio **0,9186**, exactamente el mismo valor, así que lo que se evalúa en test es realmente el modelo entrenado.

3.5 Benchmark en test: Módulo 3 vs. Módulo 4

Métrica (macro)	A · TF-IDF + NB	B · DistilBERT + LoRA	Diferencia B − A
Precision	0,8924	0,9063	+1,38 pts
Recall	0,8920	0,9015	+0,95 pts
F1-score	0,8920	0,9025	+1,04 pts
F1 weighted	0,8920	0,9025	+1,04 pts
Accuracy	0,8920	0,9015	+0,95 pts

El Transformer mejora las tres métricas del enunciado: **Precision +1,38, Recall +0,95 y F1 +1,04 puntos**. Como las clases están perfectamente balanceadas (500 por clase), el F1 macro y el weighted coinciden.

Clase	Precision A	Precision B	Recall A	Recall B	F1 A	F1 B	Δ F1 (pts)
Business	0,835	0,794	0,862	0,900	0,848	0,844	−0,49
Sci_Tech	0,870	0,905	0,870	0,862	0,870	0,883	+1,32
Sports	0,955	0,974	0,968	0,972	0,961	0,973	+1,17
World	0,910	0,952	0,868	0,872	0,888	0,910	+2,18



Matrices de confusión en test (conteo y % sobre la clase real).

Por clase, la mejora más grande está en **World (+2,18)**: la precision sube de 0,910 a 0,952 porque el modelo ya no confunde World con Sci_Tech ni con Sports. Sci_Tech y Sports también mejoran. La excepción es **Business (−0,49)**. El recall de Business sube a 0,900, pero la precision cae a 0,794, porque el modelo **sobrepredice Business**: lo asigna 567 veces, cuando hay 500. Se lleva 64 noticias de Sci_Tech (antes 45) y 46 de World (antes 36).

¿Es significativa la mejora?

	NB acierta	NB falla
LoRA acierta	1.728	**75** (las corrige el Transformer)
LoRA falla	**56** (las rompe el Transformer)	141 (fallan los dos)

El test de McNemar compara solo las noticias en las que los modelos no coinciden: 75 a favor del Transformer y 56 a favor del baseline. Da **p = 0,115**, así que **con 2.000 noticias no alcanza para afirmar que B es mejor al 5%**. Para tener una segunda mirada, hice un bootstrap pareado (2.000 remuestreos del test): el intervalo del 95% para la diferencia de F1 va de **-0,11 a +2,18 puntos**, y en el 96% de los remuestreos B queda por encima de A. La mejora es consistente en la dirección, pero chica comparada con el ruido de un test de este tamaño.

Dónde se equivoca cada uno

Real → predicha	Errores A	Errores B	Cambio
Sci_Tech → Business	45	64	+19
Business → World	25	12	−13
Sci_Tech → World	16	5	−11
World → Business	36	46	+10
World → Sci_Tech	16	7	−9
Sports → Sci_Tech	10	2	−8
Sci_Tech → Sports	4	0	−4
Business → Sports	5	2	−3

Titular (test)	Real	A	B (confianza)	
Airborne Cell-Phone Ban Likely to Remain for Now (Reuters)	Sci_Tech	Business	Sci_Tech (0,81)	lo corrige B
In China's changing society, psychotherapy gains acceptance	World	Business	World (0,69)	lo corrige B
Presidential winner faces 'twin deficits' battle (AFP)	World	World	Business (0,79)	lo rompe B
GAO: Outsourcing could hurt IT employment growth	Sci_Tech	Business	Business (0,91)	fallan los dos
Cricket: Tendulkar to miss test	World	Sports	Sports (0,82)	fallan los dos

El Transformer arregla casos en los que una palabra suelta engañaba a la bolsa de palabras. En *Airborne Cell-Phone Ban…* el baseline respondía Business, pero el tema es la regulación de una tecnología. Los errores que le quedan son mayormente noticias ambiguas: tercerización de IT (¿negocios o tecnología?), déficit fiscal en una elección (¿economía o política?), e incluso etiquetas discutibles, como una de cricket etiquetada World. Además, cuando el modelo acierta su confianza promedio es 0,93, y cuando falla baja a 0,75. Esa señal sirve para mandar a revisión manual los casos dudosos.

3.6 Costo vs. mejora

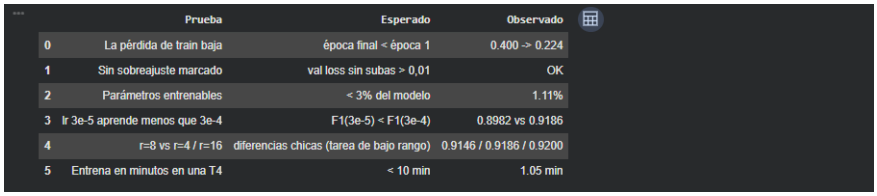
	A · TF-IDF + NB	B · DistilBERT + LoRA
F1-macro en test	0,8920	0,9025 (+1,04 pts)
Hardware	CPU	GPU Tesla T4
Tiempo de entrenamiento	0,44 s	≈ 62 s (≈ 140 veces más)
Parámetros entrenados	103.120	741.124 (1,11% de 67 M)
Inferencia de 2.000 noticias	0,09 s (+ 5,5 s de spaCy para lematizar)	1,95 s en GPU
Almacenamiento	≈ 1 MB	3,0 MB de adaptador + 268 MB del modelo base

En este dataset, cada punto de F1 me cuesta unas 140 veces más tiempo de entrenamiento, una GPU y un modelo base de 268 MB. Hay dos cosas que lo matizan. La inferencia de punta a punta no es más lenta: el baseline necesita spaCy para lematizar, y eso tarda más que el Transformer completo en GPU. Y gracias a LoRA, cada tarea nueva agrega solo un adaptador de 3 MB sobre el mismo modelo base, en lugar de 268 MB por tarea con un fine-tuning completo.

3.7 Esperado vs. observado

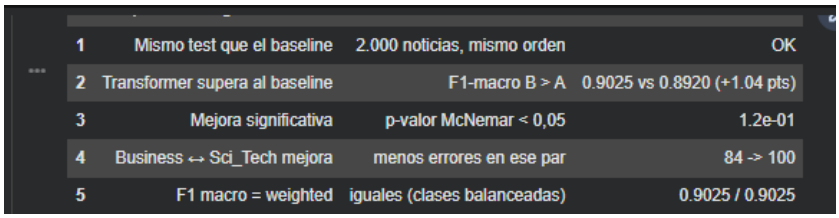
Prueba	Esperado	Observado	Resultado
Parámetros entrenables	< 3% del modelo	1,11%	Cumple
lr 3e-5 aprende menos que 3e-4	F1(3e-5) < F1(3e-4)	0,8982 vs 0,9186	Cumple
r = 4 / 8 / 16	diferencias chicas	0,9146 / 0,9186 / 0,9200	Cumple
Entrena en minutos en una T4	< 10 min	≈ 1 min (3 ejecuciones)	Cumple
Reproducible	mismas métricas	idénticas en 3 ejecuciones y al recargar	Cumple
Mismo test que el baseline	2.000 noticias, mismo orden	OK (verificado fila a fila)	Cumple

Prueba	Esperado	Observado	Resultado
El Transformer supera al baseline	F1-macro B > A	0,9025 vs 0,8920 (+1,04 pts)	Cumple
La mejora es significativa	McNemar p < 0,05	p = 0,115	No cumple
Mejora el par Business ↔ Sci_Tech	menos errores en ese par	84 → 100 (empeora)	No cumple



	Prueba	Esperado	Observado
0	La pérdida de train baja	época final < época 1	0.400 -> 0.224
1	Sin sobreajuste marcado	val loss sin subas > 0,01	OK
2	Parámetros entrenables	< 3% del modelo	1.11%
3	lr 3e-5 aprende menos que 3e-4	F1(3e-5) < F1(3e-4)	0.8982 vs 0.9186
4	r=8 vs r=4 / r=16	diferencias chicas (tarea de bajo rango)	0.9146 / 0.9186 / 0.9200
5	Entrena en minutos en una T4	< 10 min	1.05 min

Tabla de la Parte 2 en Colab.



1	Mismo test que el baseline	2.000 noticias, mismo orden	OK
2	Transformer supera al baseline	F1-macro B > A	0.9025 vs 0.8920 (+1.04 pts)
3	Mejora significativa	p-valor McNemar < 0,05	1.2e-01
4	Business ↔ Sci_Tech mejora	menos errores en ese par	84 -> 100
5	F1 macro = weighted	iguales (clases balanceadas)	0.9025 / 0.9025

Tabla de la Parte 3 en Colab.

Dejo las dos expectativas que no se cumplieron, porque también informan. La que más me sorprendió es la del par **Business ↔ Sci_Tech**. Esperaba que el contexto ayudara justo ahí, que es la frontera difusa que encontré en el Módulo 3, y pasó lo contrario. Las noticias de empresas tecnológicas son ambiguas incluso para mí al leerlas, y el modelo resolvió la duda inclinándose hacia Business.

4. Conclusiones

- **Lo que logré:** ajusté un Transformer de 67 M de parámetros entrenando solo el **1,11%**, en **alrededor de 1 minuto** de una GPU gratuita y con menos de 1 GB de VRAM. El resultado es reproducible: 3 ejecuciones con métricas idénticas y un adaptador de 3 MB que se recarga y da el mismo valor.
- **Contra mi baseline:** DistilBERT + LoRA mejora Precision, Recall y F1 en test (**0,9025 vs 0,8920**), sobre todo en World, Sci_Tech y Sports. Pero **la mejora no es estadísticamente significativa** con este test (McNemar p = 0,115; IC 95% de −0,1 a +2,2 puntos).
- **Lo que no mejoró:** Business, porque el modelo lo sobrepredice (567 de 2.000), y la confusión Business ↔ Sci_Tech, que es más ambigüedad del dato que falta de capacidad.
- **Costo:** unas 140 veces más tiempo de entrenamiento y una GPU para +1 punto. Con solo 7.200 noticias, un TF-IDF bien armado ya captura casi todo lo que hay: los temas de AG News se reconocen en gran parte por palabras clave. Si solo importara el costo, me quedaría con el baseline; si importa cada punto, o se van a sumar más tareas sobre el mismo modelo base, LoRA es la opción que escala.
- **Próximos pasos:** (1) entrenar con el AG News completo (120.000 noticias), donde el Transformer debería sacar más ventaja; (2) roberta-base, que ya dio 0,926 en validación; (3) corregir el sesgo hacia Business con pesos por clase o ajustando el umbral; (4) usar la confianza del modelo para derivar a revisión los casos ambiguos.

Código, notebooks ejecutados, adaptador y CSVs con todas las métricas: github.com/scarletluque0805/ds3-transformer-lora (Parte 1: datos y configuración · Parte 2: fine-tuning y ablación · Parte 3: benchmark). Las versiones de librerías usan rangos y están probadas para Python 3.10 a 3.13.